

Computer System Modeling and Analysis

Lab Project (20 marks)

1. Project Overview

In this project you will build a complete Discrete Event Simulation (DES) of a queuing system entirely from scratch. You are not given fixed data — instead you generate random values, implement the full simulation engine, and compute key performance metrics.

The simulation models the classic single-queue / multi-server system, where customers arrive, possibly wait in a queue, get served, and leave. You will extend it step by step across six parts.

2. Part 1 — Random Value Generation [2 marks]

You must generate inter-arrival times and service times programmatically. Hardcoded values receive zero marks for Parts 1–6.

4.1 Linear Congruential Generator (LCG)

Use the following formula:

$$X_{n+1} = (a \times X_n + c) \bmod m$$

where $a = 1664525$, $c = 1013904223$, $m = 2^{32} = 4,294,967,296$

To get a value in the range $[1, 10]$:

$$\text{value} = (X_{n+1} \bmod 10) + 1$$

Example trace (seed = 42):

Xn (current)	Formula step	Xn+1	Xn+1 / m	Value [1,10]
X0 (seed)	$a \times X_0 + c \bmod m$	X1	X_1 / m	Scaled [1,10]
42	$1664525 \times 42 + 1013904223$	1015614293	0.2364...	3
1015614293	$1664525 \times 1015614293 + \dots$	2837248318	0.6604...	7
2837248318	...	749823501	0.1746...	2

4.2 Requirements

- Allow the user to input n (number of customers) and a seed.
- Generate n inter-arrival times and n service times using the LCG.
- Print the generated sequences before running the simulation.
- Running the same seed must always produce the same results (reproducibility).
-

3. Part 2 — Single-Server Simulation [5 marks]

Simulate a single-server FCFS (First-Come-First-Served) queue and produce the full event table

Output Requirements

- Print the complete event table (Arrival #, Arrival Time, Begin Service, End Service, System Wait, Queue Wait).
- Print W , Q , and U clearly labelled.
- Verify against the lab example ($W = 3.875$, $Q = 0.357$) using the given data.

4. Part 3 — Multi-Server Simulation [5 marks]

Extend your simulation to support s servers (user inputs s). Customers go to the first available server.

Requirements

- Prompt the user to enter the number of servers (try $s = 1, 2$, and 3).
- Record which server each customer was assigned to.
- Compute W , Q , and per-server utilization for each value of s .
- Display the following comparison table in your report.

Servers	Avg Wait W	Queue Wait	Q (queue)	Utilization	Comment
1 server	High	High	High	Low	Baseline
2 servers	Lower	Lower	Lower	Higher	Better
3 servers	Lowest	Lowest	Lowest	Highest	Best (diminishing returns)

Note: With more servers, W and Q decrease but each server becomes less utilized (more idle time). This is the classic cost-performance trade-off.

5. Part 4 — Queue Discipline [3 marks]

In addition to FCFS, implement one alternative discipline and compare the results.

Option A — LCFS (Last-Come-First-Served)

When the server becomes free, serve the most recently arrived waiting customer instead of the earliest.

- Implement the waiting queue as a stack (push on arrival, pop on service).
- Compute W and Q under LCFS and compare with FCFS using the same random data.
- Comment on the difference: does the average wait change? Does individual fairness change?

Option B — Priority Queue

Assign each customer a random priority (1 = highest, 3 = lowest). Serve the highest-priority customer first.

- On tie in priority, use FCFS as tiebreaker.
- Report average W and Q broken down per priority level.
- Comment: which priority class benefits most? Which suffers?

Tip: Only implement ONE option (A or B). Both options are worth the same marks.

6. Part 5 — Warm-up Period [2 marks]

8.1 Why Is Warm-up Needed?

When the simulation starts, all servers are idle and the queue is empty. This is called the transient state and it does not represent normal system operation. Statistics collected during this phase produce optimistically low estimates of W and Q .

Example: In a real bank, customers arrive throughout the day and there are always some people waiting. But your simulation begins at time zero with nobody — so the first few customers are served immediately with zero queue time. This pulls the average down artificially.

The solution is to run the simulation for k extra customers at the start (the warm-up period) and only start recording statistics after those k customers have been processed. The system reaches its steady state during the warm-up.

Requirements

- Allow the user to specify k (the warm-up count, default 0).

- The simulation must still run the warm-up customers through the system — they just do not contribute to W, Q, or U.
- In your report, run the same data twice: once with $k = 0$ and once with $k = 3$ or $k = 5$.
- Fill in the following comparison table and briefly comment on the difference:

Metric	Without warm-up	With warm-up ($k=3$)	Difference
Avg System Wait W	Lower (optimistic)	Higher (realistic)	W increases
Avg Queue Length Q	Lower	Higher	Q increases
Server Utilization	Similar	Similar	Minimal change

Note: Expected observation: W and Q are slightly higher with warm-up because the artificially low initial measurements are excluded. The difference is more noticeable with small n.

7. Part 6 — Visualization [3 marks]

Generate at least two charts. They may be produced by any library (matplotlib, Excel, etc.) and must appear in your PDF report.

Required — Gantt Chart

Show the busy and idle intervals for every server over time. Each server is one horizontal row. Coloured bars represent service, gaps represent idle time. Label each bar with the customer number.

- X-axis: simulation time (0 to total_time).
- Y-axis: one row per server (Server 1, Server 2, ...).
- Use a different colour for each server.

Choose One Additional Chart

- Waiting time bar chart — one bar per customer showing their system wait.
- Queue length over time — line chart showing how many customers are waiting at each time unit.
- Server utilization comparison — grouped bar chart for $s = 1, 2, 3$.
- W vs n chart — how does average wait change as n grows from 10 to 100?

8. Sample Run Output (just a sample!!!!)

Your program's console output should look similar to the following when given $n = 8$, seed = 42, 1 server, FCFS:

```
=====
      DISCRETE EVENT SIMULATION - SAMPLE RUN OUTPUT
=====

Configuration:
  Customers (n)      : 8
  Seed               : 42
  Servers            : 1
  Discipline         : FCFS
  Distribution       : Uniform [1,10]
  Warm-up (k)       : 0
  Max queue length   : Unlimited
-----

Inter-arrival times : [1, 4, 2, 1, 8, 2, 4, 3]
Service times       : [2, 5, 4, 1, 3, 2, 1, 3]
-----

#   Arrive  Begin  End   Server SysWait  QWait
1    1      1     3     S1      2      0
2    5      5    10     S1      5      0
3    7     10    14     S1      7      3
4    8     14    15     S1      7      6
5   16     16    19     S1      3      0
6   18     19    21     S1      3      1
7   22     22    23     S1      1      0
8   25     25    28     S1      3      0
-----

RESULTS
  Average system waiting time  W  = 3.875 min
  Average queue length        Q  = 0.357 customers
  Server utilization          U  = 78.6 %
  Total simulation time       = 28 time units
  Customers rejected          = 0
=====
```

Note: Your exact numbers will differ based on your random generator. The sample above uses the fixed lab data (inter-arrival: 1 4 2 1 8 2 4 3 | service: 2 5 4 1 3 2 1 3) to match the known answer $W = 3.875$.

9. Deliverables

Submit a single ZIP file TeamLeaderID.zip containing:

#	Item	Status
1	Source code — well-commented, all 6 parts implemented	Required
2	PDF report (1–3 pages) — sample output, charts, comparisons	Required
3	README.txt — how to install dependencies and run the code	Required

The PDF report must include:

- Sample console output for $n = 20$ customers.
- The Gantt chart and one other chart (Part 6).
- The multi-server comparison table (Part 3).
- The warm-up comparison table (Part 5).
- A short paragraph (3–5 sentences) commenting on your observations.

10. Constraints and Rules

- Any programming language is accepted (Python, Java, C++, C#, etc.).
- Simulation frameworks such as SimPy, GPSS, AnyLogic, and Arena are NOT allowed.
- Hardcoded inter-arrival or service times receive zero for Parts 1–6.
- Plagiarism results in zero for the entire project.
- Late submissions lose 2 marks per day.

11. Grading Rubric

Part	Description	Marks
Part 1	Random Value Generation — LCG with seed, inter-arrival & service times	2
Part 2	Single-server simulation — full table, W , Q , server utilization	5
Part 3	Multi-server simulation — 2+ servers, comparison table	5
Part 4	Queue discipline — LCFS or Priority Queue + FCFS comparison	3
Part 5	Warm-up period — skip first k customers, compare W and Q	2
Part 6	Visualization — Gantt chart + at least one other chart	3
TOTAL		20

Best Output: I love it

SIMULATION PARAMETERS

Number of customers (n)

10

Number of servers

1

2

3

Distribution type

Uniform

Exponential

Max queue length (0 = unlimited)

Random seed

43

Queue discipline

FCFS

LCFS

Priority

Warm-up customers (skip stats)

1

Metrics

Table

Gantt chart